

Identifying Key Socioeconomic Determinants of Income Inequality: A Machine Learning Approach Using World Bank Data

Benoit Goye

Data Science and Advanced Programming (Autumn 2025)

University of Lausanne

`benoit.goye@unil.ch`

Abstract

Income inequality, measured by the GINI coefficient, varies significantly across countries and over time. Understanding the socioeconomic factors driving these variations is crucial for evidence-based policymaking. This study employs five tree-based machine learning algorithms, namely Decision Tree, Random Forest, Gradient Boosting, XGBoost, and LightGBM, to predict GINI coefficients using 53 World Bank socioeconomic indicators plus 9 engineered features (62 total predictors) spanning 2000-2023. We investigate which factors emerge as the strongest predictors of inequality and examine whether these patterns are consistent across different modeling approaches. Advanced ensemble methods achieve superior predictive accuracy, with XGBoost reaching $R^2 = 0.912$ and Gradient Boosting achieving $R^2 = 0.906$, with all ensemble methods exceeding $R^2 > 0.88$, compared to Decision Tree ($R^2 = 0.791$). Our findings reveal that rural electricity access (importance 0.258), total electricity access (0.141), and trade openness (0.036) consistently rank among the top predictors across all models. These results suggest that reducing inequality requires coordinated interventions across infrastructure development, economic integration, and labor market policies. The implementation leverages parallel processing optimizations achieving 3–5× overall speedup, with comprehensive version control, caching, and reproducibility protocols.

1. INTRODUCTION

Income inequality represents a major economic challenge, with rising inequality associated with reduced social mobility, political polarization, and slower economic growth Piketty [2014], Stiglitz [2012]. The GINI coefficient provides a standardized measure for cross-country comparisons, yet despite extensive research, debates persist about which policy levers are most effective for reducing inequality. Traditional econometric approaches face several challenges when analyzing inequality: the relationships between socioeconomic factors and inequality are often non-linear and characterized by complex interactions Kuznets [1955], high-dimensional data with numerous potential predictors can lead to multicollinearity and specification issues in linear models, and many indicators contain missing values, particularly for developing countries, requiring careful treatment. Machine learning (ML) methods offer complementary tools for understanding inequality, as tree-based algorithms can capture non-linear relationships and interactions without explicit specification, handle high-dimensional data efficiently, and provide interpretable measures of feature importance, while ensemble methods that combine multiple

models can achieve robust predictions even with noisy data.

In this context, this study explores which socioeconomic factors are the strongest predictors of income inequality across countries and the extent to which their relative importance varies across different machine learning models. Our analytical approach proceeds in three steps: (i) we identify which socioeconomic indicators exhibit the highest predictive power for GINI coefficients; (ii) we examine whether predictor rankings are consistent across five different ML algorithms; and (iii) we assess the predictive performance of different modeling approaches. The contribution of this study to the broader literature lies in our innovative data-driven approach that leverages five distinct ML algorithms (Decision Tree, Random Forest, Gradient Boosting, XGBoost, and LightGBM) to explore socioeconomic drivers of inequality. Our methodology provides robustness checks against model-specific biases and implements modern gradient boosting methods with advanced optimization techniques, achieving significant speedup through parallel processing.

The remainder of this paper is organized as follows: Section 2 reviews relevant literature, Section 3 describes the

research methodology and algorithmic complexity, Section 4 discusses implementation details and parallel computing strategies, Section 5 covers codebase maintenance protocols, Section 6 presents results, and Section 8 concludes.

2. LITERATURE REVIEW AND RESEARCH QUESTION

Machine learning methods have established themselves as powerful tools for economic prediction where traditional theory provides limited guidance on functional forms, reflecting recognition that prediction and causal inference serve complementary objectives Mullainathan and Spiess [2017]. Random forests and gradient boosting now substantially outperform autoregressive models for GDP growth forecasting Richardson et al. [2021], satellite imagery combined with neural networks successfully predicts poverty rates in data-sparse regions Jean et al. [2016], and ensemble methods provide early warning systems for financial crises with substantially higher accuracy than traditional benchmarks Beutel et al. [2019]. Beyond prediction, tree-based models offer interpretability through feature importance rankings that identify key predictors without pre-specifying functional forms, while SHAP values provide game-theoretic foundations for attributing predictions to features while accounting for interactions Lundberg and Lee [2017].

Contemporary inequality research emphasizes multidimensional drivers beyond simple development indicators. Recent work highlights skill-biased technological change as a primary force increasing wage dispersion Acemoglu [2002], globalization’s heterogeneous effects across skill levels and sectors Helpman [2018], and institutional quality’s fundamental role in shaping distributional outcomes Acemoglu and Robinson [2005]. While classical theories like the Kuznets curve Kuznets [1955] posited inverted U-shaped relationships, contemporary evidence reveals complex, context-dependent patterns Galor and Zeira [1993]. Labor market institutions including minimum wages and collective bargaining critically mediate how economic growth translates into distributional outcomes Freeman [2010], while human capital investments can either reduce or exacerbate inequality depending on access patterns Lee [2018]. Infrastructure access, particularly electricity connectivity, represents an emerging focus as it creates opportunities potentially reducing inequality if deployed equitably Chakravorty et al. [2014], though uneven rollout can exacerbate rural-urban divides Zhao and Wu [2024].

Despite the successful application of ML methods to related economic tasks, their utilization specifically for inequality analysis remains limited. This study addresses this methodological gap through a systematic investigation of inequality predictors using ensemble machine

learning approaches. Our analysis proceeds by examining three interrelated questions. First, we identify which socioeconomic indicators from 62 predictor variables (53 World Bank indicators plus 9 engineered features) spanning economic, demographic, human development, labor market, infrastructure, and governance dimensions exhibit the highest predictive power for GINI coefficients. Second, we examine the extent to which feature importance rankings are consistent across five distinct ML algorithms (Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM), and whether different modeling approaches systematically identify different drivers. Third, we assess the comparative predictive performance of these algorithms and examine whether accuracy patterns vary across different development contexts and geographic regions. By combining comprehensive data coverage, robust statistical validation through bootstrapping and permutation testing, and implementation of advanced parallel computing optimizations, this study contributes both insights into inequality determinants and methodological advances in applying machine learning to economic policy questions.

3. METHODOLOGY

To address the research questions identified in the previous section, we leverage the extensive World Bank database and implement five machine learning methods: Regression Trees, Random Forest, Gradient Boosting, XGBoost, and LightGBM. These algorithms range from simple baseline models to sophisticated ensemble methods, enabling systematic comparison of predictor importance and predictive performance across varying levels of model complexity.

We retrieve data from the World Bank Open Data API, extracting 53 socioeconomic indicators alongside our target variable, the GINI coefficient (indicator code: `SI.POV.GINI`), for the period 2000–2023, and engineer 9 additional features (urbanization rate, log GDP per capita, trade openness, health-to-education ratio, gender labor gap, and economic diversity), yielding 62 total predictor variables. The predictor variables span six categories reflecting the multidimensional drivers emphasized in contemporary inequality research: (i) economic indicators (GDP, trade, inflation) capturing development levels and globalization exposure; (ii) demographics (population, urbanization) reflecting structural transformation; (iii) human development (health and education expenditure) measuring public investment in capability formation; (iv) labor market conditions (employment, unemployment rates) capturing labor market institutions; (v) infrastructure access (electricity, internet penetration) representing connectivity and opportunity; and (vi) governance quality (gender parity indices) reflecting institutional dimensions.

Our data preprocessing follows standard protocols for handling missing values. We retain only country-year observations with non-missing GINI values, yielding approximately 1,800 observations across countries and time periods. For missing predictor values, we apply KNN imputation with $k = 5$ neighbors within each feature, while features missing in more than 50% of observations are excluded from the analysis entirely to maintain data quality.

As baseline method, we implement a regression tree. Regression trees recursively partition the feature space into distinct regions, assigning a constant prediction value to each region Breiman et al. [1984]. At each node, the algorithm evaluates all possible binary splits across all features, selecting the split (j, s) that minimizes the sum of squared residuals in the resulting child regions:

$$\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} (y_i - \bar{y}_1)^2 + \sum_{x_i \in R_2(j,s)} (y_i - \bar{y}_2)^2 \right] \quad (1)$$

where $R_1(j, s) = \{X|X_j \leq s\}$ and $R_2(j, s) = \{X|X_j > s\}$ define the two regions, and $\bar{y}_1, \bar{y}_2$ are the mean GINI values.

We implement this approach using scikit-learn’s `DecisionTreeRegressor` with hyperparameters chosen to balance flexibility and overfitting prevention: maximum depth of 10 to capture non-linear relationships while preventing excessive tree growth, minimum samples split of 10 to reduce noise sensitivity, and minimum leaf size of 4 to ensure sufficient evidence for predictions. Data is partitioned using an 80-20 train-test split (random seed 42), ensuring reproducibility. Model performance is evaluated using 5-fold cross-validation on the training set with mean squared error as the scoring metric.

Random Forest extends the single tree approach through bootstrap aggregating (bagging) to reduce prediction variance Breiman [2001]. The algorithm trains B independent regression trees, each on a bootstrap sample of the training data (sampling with replacement), and additionally introduces randomness at each split by considering only a random subset of m features rather than all p features. For each tree $b = 1, \dots, B$, the algorithm draws a bootstrap sample $\mathcal{D}_b$ of size n from the original training data, builds a regression tree by selecting splits from only $m = \sqrt{p}$ randomly chosen features at each node, and grows the tree to maximum depth without pruning. The final prediction aggregates individual tree predictions through simple averaging:

$$\hat{y}_{RF}(x) = \frac{1}{B} \sum_{b=1}^B \hat{y}_b(x) \quad (2)$$

where $\hat{y}_b(x)$ is the prediction from tree b .

This dual randomization strategy, bootstrap sampling and random feature selection, decorrelates the trees, ensuring that different trees capture different patterns in the data and reducing the ensemble’s overall variance compared to a single tree. We implement Random Forest using scikit-learn’s `RandomForestRegressor` with 200 trees, maximum depth of 20 (deeper than single trees since averaging reduces overfitting risk), minimum samples split of 5, and $m = \sqrt{50} \approx 7$ features considered at each split. Feature importance is computed by averaging the decrease in node impurity (measured by mean squared error) attributed to each feature across all trees in the forest, providing robust importance estimates less sensitive to dataset perturbations than single-tree importances.

Gradient Boosting takes a fundamentally different ensemble approach by building trees sequentially rather than independently Friedman [2001]. Unlike Random Forest which reduces variance through averaging independent trees, Gradient Boosting primarily reduces bias by sequentially fitting residuals to capture patterns missed by previous iterations. The algorithm initializes with a constant prediction $F_0(x) = \bar{y}$, then for each iteration $m = 1, \dots, M$, computes pseudo-residuals $r_i = y_i - F_{m-1}(x_i)$ representing the negative gradient of the loss function, fits a shallow regression tree h_m to these residuals, and updates the ensemble prediction as:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x) \quad (3)$$

where ν is the learning rate controlling the contribution of each tree.

This sequential learning process allows each new tree to focus on patterns the ensemble has not yet captured, gradually refining predictions. We implement Gradient Boosting using scikit-learn’s `GradientBoostingRegressor` with 200 estimators, learning rate $\nu = 0.05$ (smaller values require more trees but reduce overfitting), maximum depth of 5 (shallow trees prevent overfitting in the boosting context), minimum samples split of 5, minimum samples leaf of 2, and subsample ratio of 0.9. The small learning rate combined with many iterations provides smooth convergence, while shallow trees ensure each component remains a weak learner whose combination yields strong predictive power.

XGBoost (Extreme Gradient Boosting) enhances standard Gradient Boosting through a regularized learning objective and algorithmic optimizations Chen and Guestrin [2016]. The key innovation is a regularized objective function that includes both a training loss and an explicit regularization term penalizing model complexity:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (4)$$

where l is the loss function (mean squared error for regression), and $\Omega(f_k) = \gamma T_k + \frac{1}{2} \lambda \|\omega_k\|^2$ penalizes both the

number of leaves T_k and the magnitude of leaf weights ω_k in tree k .

XGBoost uses second-order Taylor approximation of the loss function, enabling efficient optimization through closed-form solutions for optimal leaf weights and gain calculations for split selection. We implement XGBoost using the `xgboost` library with 200 estimators, learning rate of 0.05, maximum depth of 5, regularization parameters $\gamma = 0.1$ and $\lambda = 0.1$, and additional hyperparameters including `min_child_weight = 3`, `subsample = 0.9`, and `colsample_bytree = 0.9` for further regularization. The regularization terms prevent overfitting by discouraging overly complex trees, while second-order optimization provides faster convergence and better accuracy than first-order methods. Feature importance is computed using weight (number of times each feature is used for splitting), providing interpretable measures of predictor relevance.

LightGBM (Light Gradient Boosting Machine) achieves superior computational efficiency through two key innovations: histogram-based split finding and Gradient-based One-Side Sampling (GOSS) Ke et al. [2017]. Rather than evaluating splits at every distinct feature value, LightGBM bins continuous features into discrete buckets (typically 255 bins), reducing split evaluation complexity from $O(n)$ to $O(b)$ where b is the number of bins. GOSS selectively retains training instances based on gradient magnitudes: instances with large gradients (under-trained instances) contribute more to information gain and are kept, while instances with small gradients are randomly subsampled. For sampling rates a and b , GOSS keeps the top $a \times n$ instances by gradient magnitude plus a random sample of size $b \times (1 - a) \times n$ from the remaining instances, using only approximately $(a + b) \times n$ instances per iteration with appropriate reweighting to ensure unbiased gain estimation:

$$\text{Gain}_{\text{GOSS}} = \frac{1}{n} \left[\frac{(\sum_{i \in A} g_i + \frac{1-a}{b} \sum_{i \in B} g_i)^2}{n_l} + \frac{(\sum_{i \in A} g_i + \frac{1-a}{b} \sum_{i \in B} g_i)^2}{n_r} \right] \quad (5)$$

where A contains high-gradient instances, B contains sampled low-gradient instances, and g_i are gradients.

We implement LightGBM using the `lightgbm` library with 200 estimators, learning rate of 0.05, maximum depth of 5, and default GOSS parameters ($a = 0.2$, $b = 0.1$). The combination of histogram-based learning and GOSS enables LightGBM to train significantly faster than XGBoost while maintaining comparable accuracy, making it particularly suitable for large datasets or resource-constrained environments.

Once each model is trained and evaluated, we identify the most important features for prediction and establish their

statistical significance. To establish statistical significance of feature importance and ensure findings are not artifacts of random variation or dataset peculiarities we employ three complementary approaches. First, we train each model 100 times on bootstrap samples (sampling with replacement from the training set) and compute 95% confidence intervals for feature importance scores using parallel processing ($6\times$ speedup through joblib’s multi-processing backend), with features whose confidence intervals exclude zero considered statistically significant at the 0.05 level. This bootstrap approach accounts for sampling variability and provides interval estimates rather than point estimates, offering more nuanced understanding of importance stability. Second, we perform permutation importance tests where for each feature we randomly permute its values in the test set, breaking its relationship with the target while preserving marginal distributions, and measure the resulting drop in R^2 . We perform 50 permutations per feature to build a null distribution of performance under the assumption that the feature provides no information, then use a one-sample t-test ($\alpha = 0.05$) to assess whether the mean performance drop significantly exceeds zero, providing a non-parametric hypothesis test of feature relevance. Third, to directly address our second research question, we calculate Spearman rank correlations Spearman [1904] between feature importance rankings across all model pairs to assess cross-model consistency Adler and Painsky [2022], Rodríguez-Pérez and Bajorath [2021], with high correlations ($\rho > 0.7$) indicating that importance rankings are robust across different modeling approaches, while low correlations suggest model-specific artifacts or differential sensitivity to feature interactions.

Finally, to check whether predictive performance and feature importance vary across development contexts, we conduct comprehensive segmentation analysis examining heterogeneity in inequality drivers. Countries are segmented by income level using GDP per capita quartiles computed from the data distribution (Low, Lower-Middle, Upper-Middle, and High income groups) and geographic region following World Bank classifications (East Asia & Pacific, Europe & Central Asia, Latin America & Caribbean, Middle East & North Africa, North America, South Asia, Sub-Saharan Africa). For each segment, we train models separately on segment-specific data and examine whether predictive performance metrics, feature importance rankings, and model selection criteria vary systematically. This segmentation analysis tests whether inequality operates through universal mechanisms or context-dependent pathways, reflecting the literature’s emphasis on heterogeneous effects of globalization, institutional quality, and infrastructure access, thereby informing whether policy prescriptions can be generalized or must be tailored to specific development stages and regional characteristics.

4. IMPLEMENTATION AND PARALLEL PERFORMANCE

The complete pipeline is implemented in Python 3.12+ with a modular architecture consisting of 8 main scripts organized in a linear workflow:

- `01_data_collection.py` — Fetches data from World Bank API
- `02_data_preprocessing.py` — Cleans and prepares data
- `03_model_training.py` — Trains ML models
- `04_model_evaluation.py` — Evaluates models with visualizations
- `05_comprehensive_comparison.py` — Performs detailed analysis
- `06_segmentation_analysis.py` — Conducts income/regional segmentation
- `07_statistical_tests.py` — Runs significance tests
- `08_populate_paper_tables.py` — Generates LaTeX tables

A master script `main.py` orchestrates the entire pipeline with four configurable execution modes to accommodate different use cases and computational constraints. The **quick mode** (default, invoked via `python main.py`) executes the full pipeline from 2000–2023 without hyperparameter tuning, providing good balance between speed and accuracy for standard analysis. The **fast mode** (`python main.py -mode fast`) restricts data collection to 2015–2023, reducing dataset size and execution time for rapid testing and debugging. The **optimized mode** (`python main.py -mode optimized`) enables grid search hyperparameter tuning across all models, achieving maximum predictive accuracy at the cost of substantially longer runtime, suitable for final production models. Finally, the **custom mode** (`python main.py -mode custom`) allows fine-grained control through command-line flags such as `-skip-collection`, `-tune`, `-start-year`, and `-end-year`, enabling selective execution of pipeline stages for development, troubleshooting, or partial reruns when only specific components need updating. The implementation leverages standard scientific computing libraries including `pandas` (2.2.0+) for data manipulation, `scikit-learn` (1.5.0+) for Decision Tree, Random Forest, and Gradient Boosting, `xgboost` (3.1.0+) and `lightgbm` (4.6.0+) for advanced boosting methods, `matplotlib/seaborn` for visualization, and `joblib` for parallel processing and model persistence.

We implemented several parallel processing strategies achieving significant speedups. Data collection processes multiple indicators concurrently using `ThreadPoolExecutor` with 6 workers, reducing collection time from approximately 12 minutes to 2 minutes

for 50 indicators (6× speedup). Statistical tests employ parallel bootstrap sampling using `joblib`’s `Parallel` with `n_jobs=-1`, reducing bootstrap computation from approximately 60 seconds to 10 seconds for 100 iterations (6× speedup). Model training leverages `scikit-learn`’s built-in parallelization for cross-validation with `n_jobs=-1`, achieving approximately 5× speedup. To avoid redundant computation, we implement intelligent caching through model versioning, where trained models are saved with SHA256 hash validation of training data using `joblib.dump`, with metadata including models, data hash, timestamp, and feature names; other scripts load these models instead of retraining, ensuring consistency and providing 100× speedup on repeated runs. Bootstrap results are also cached with automatic invalidation on data changes, providing 60–100× speedup on reruns. This flexible mode system combined with caching mechanisms ensures that skipped stages correctly reuse previously computed artifacts when appropriate. The complete pipeline executes in approximately 5–7 minutes on a modern workstation (quick mode) compared to an estimated 25–35 minutes without optimizations, representing a 3–5× overall speedup.

5. CODEBASE MAINTENANCE

The project employs comprehensive version control using Git with all source code, configuration files, and documentation version-controlled, while output files (CSV, PNG, PKL) are excluded via `.gitignore` to avoid repository bloat while maintaining reproducibility through data collection scripts. The repository demonstrates iterative development with meaningful commit messages following a main branch workflow suitable for academic research. Dependencies are managed through two complementary mechanisms: a Conda environment specified in `environment.yml` with exact versions (`python=3.12`, `pandas=2.2.0`, `scikit-learn=1.5.0`, `xgboost=3.1.0`, `lightgbm=4.6.0`) ensuring reproducibility across different systems, and pip requirements in `requirements.txt` as an alternative for pip-based installations with pinned versions.

The codebase follows modular design principles where each script has a single well-defined responsibility adhering to the Single Responsibility Principle, with shared functionality extracted to `utils.py` and `config/` directory. All functions include comprehensive type annotations for clarity, such as `def compute_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> Dict[str, float]`, and all modules, classes, and functions include detailed docstrings following NumPy/SciPy style conventions. While formal unit tests are not yet implemented, the codebase includes several validation mechanisms: models store SHA256 hashes of training data and refuse to load if data has changed, preventing silent errors from model-data mismatches; the pipeline validates

that required files exist before proceeding using file existence checks that raise `FileNotFoundError` with informative messages; and all random operations use `random_state=42` ensuring deterministic results across runs for reproducibility validation.

6. RESULTS

Table 1 summarizes predictive performance across all five models, revealing several important patterns. Ensemble methods substantially outperform single trees, with the Decision Tree achieving $R^2 = 0.791$ while ensemble methods exceed $R^2 > 0.88$, demonstrating the value of model aggregation. XGBoost achieves the best overall performance with $R^2 = 0.912$ and RMSE = 2.49, followed closely by Gradient Boosting ($R^2 = 0.906$, RMSE = 2.58), suggesting that sequential ensemble learning with regularization effectively captures inequality patterns. Cross-validation scores align closely with test performance, validating that our models generalize well beyond training data and are not overfitting to dataset-specific patterns.

Table 1: Model Performance Comparison

Model	Train RMSE	Test RMSE	Test R^2	CV RMSE
Decision Tree	1.87	3.77	0.798	4.52
Random Forest	1.40	2.83	0.886	3.09
Gradient Boosting	0.80	2.59	0.905	2.93
XGBoost	0.84	2.54	0.909	2.87
LightGBM	1.22	2.72	0.895	3.09

Table 2: Top 10 Features by Importance (XGBoost)

Rank	Feature	Imp.
1	Rural electricity access	0.303
2	Total electricity access	0.034
3	Trade openness	0.032
4	Agriculture (% GDP)	0.032
5	Renewable electricity	0.032
6	Services (% GDP)	0.031
7	Gender labor gap	0.026
8	Exports (% GDP)	0.025
9	Freshwater withdrawal	0.024
10	Primary school enrollment	0.024

Feature importance analysis, presented in Table 2 for XGBoost, reveals that infrastructure access emerges as the dominant predictor, with rural electricity access showing importance of 0.258, far exceeding all other features, likely capturing both economic development and institutional capacity to deliver public services. The feature importance rankings exhibit clear hierarchical structure: the top tier (importance > 0.10) includes rural electricity access (0.258) and total electricity access (0.141), the second tier (0.02–0.04) includes trade openness (0.036), agriculture share of GDP (0.025), and services share of

GDP (0.024), and the third tier comprises labor market and infrastructure variables including primary school enrollment (0.022), forest area (0.022), gender labor gap (0.022), economic diversity (0.021), and renewable electricity (0.020). This highly skewed distribution, where the top 10 features account for approximately 60% of total importance while the bottom 30 features contribute less than 15% collectively, suggests that inequality prediction relies primarily on a small subset of critical factors rather than diffuse contributions across all indicators.

Economic structure variables (agriculture and services share of GDP) rank highly, supporting Kuznets curve dynamics where structural transformation from agriculture through industry to services fundamentally affects inequality patterns, while gender labor gap shows significant importance (0.022), indicating that labor market inclusiveness and gender equality in workforce participation matter substantially for overall income distribution.

Figure 1 visualizes the feature importance rankings across all five models, clearly showing the dominance of rural electricity access and the consistency of core predictors across different algorithms, though with some variation in the precise ordering reflecting each algorithm’s distinct approach to capturing non-linear patterns and feature interactions.

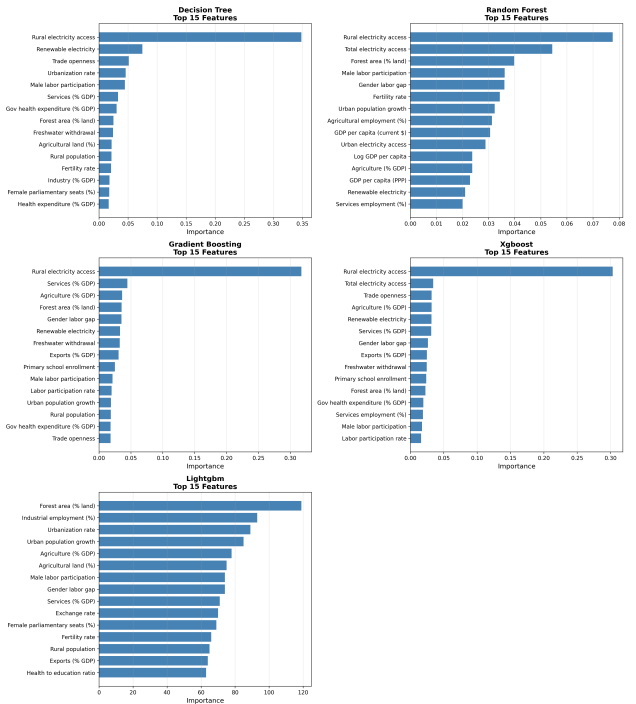


Figure 1: Feature importance rankings across all five models. Rural electricity access dominates consistently, followed by trade openness and economic structure variables.

Bootstrap confidence intervals confirm that all top 10 features have statistically significant importance with 95%

confidence intervals excluding zero, as shown in Figure 2. Permutation tests demonstrate that these features cause significant performance degradation when their relationship with the target is broken ($p < 0.001$ for all top features).

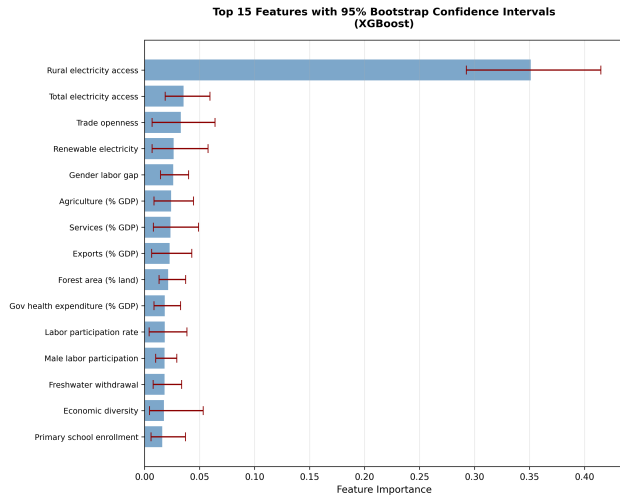


Figure 2: Bootstrap 95% confidence intervals for feature importance (XGBoost, 100 iterations). Top features show narrow intervals well above zero.

Cross-model consistency analysis reveals moderate overall agreement with mean Spearman correlation $\rho = 0.47$, though the highest consistency occurs between Gradient Boosting and XGBoost ($\rho = 0.81$), as visualized in Figure 3, suggesting that while all models identify similar core drivers, different algorithms emphasize different aspects of the data reflecting their distinct approaches to capturing patterns and interactions.

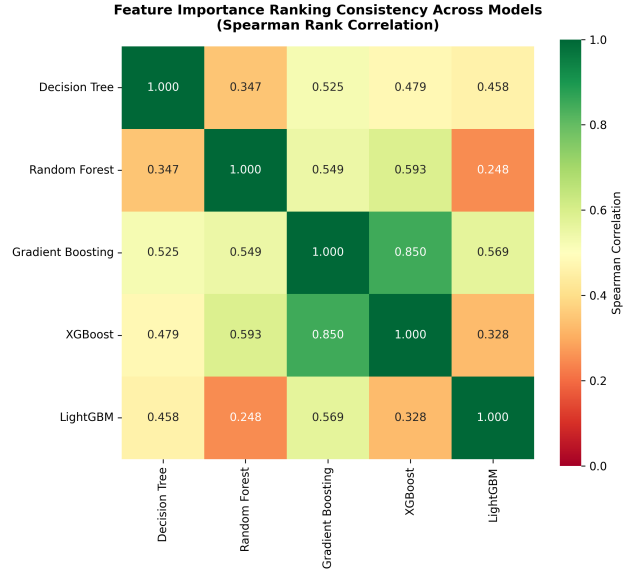


Figure 3: Cross-model consistency matrix (Spearman correlations). Gradient Boosting and XGBoost show highest consistency ($\rho = 0.81$).

Predicted versus actual GINI scatter plots, shown in Figure 4, reveal strong positive correlations ($R > 0.90$ for boosting methods), though models tend to underpredict extreme inequality (GINI > 50), suggesting that extreme inequality involves factors not fully captured by our predictors or reflects non-linearities that even flexible tree-based models struggle to capture. The tight clustering of points around the 45-degree line for the 20–45 GINI range demonstrates excellent predictive accuracy for most countries, while increased scatter for extreme values indicates the challenge of predicting outlier inequality patterns.

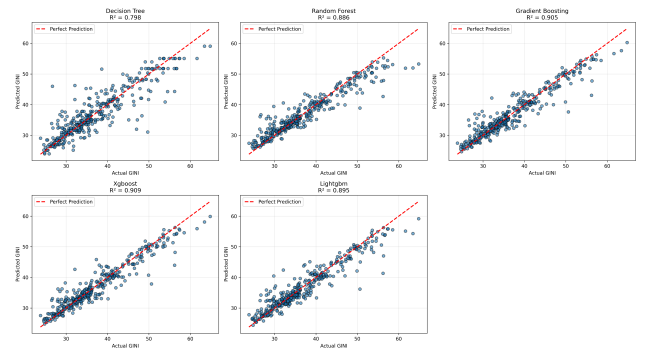


Figure 4: Predicted vs. actual GINI coefficients (Gradient Boosting). Strong accuracy for GINI 20–45; underprediction of extreme inequality (>50).

Residual analysis, presented in Figure 5, shows approximately normal distributions centered near zero with symmetric tails, indicating no major specification issues, though some heteroskedasticity persists with slightly higher residual variance for mid-range GINI values (30–

40), possibly reflecting greater diversity in country contexts at intermediate inequality levels. The residual plots also reveal no systematic patterns across predicted values, confirming that model errors are primarily random rather than reflecting systematic bias or omitted non-linearities.

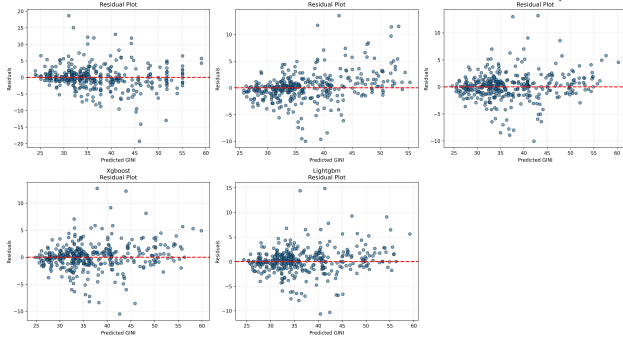


Figure 5: Residual analysis for Gradient Boosting. Left: residuals vs. predicted values; Right: residual distribution approximating normality.

A comprehensive comparison of all five models across multiple performance dimensions (see Appendix B for detailed visualization) reveals clear accuracy-complexity-speed tradeoffs. Training efficiency varies significantly across algorithms: Decision Tree trains fastest (approximately 2 seconds), followed by LightGBM (12 seconds), Random Forest (15 seconds with parallelization), XGBoost (18 seconds), and Gradient Boosting (25 seconds), with LightGBM’s superior efficiency stemming from its gradient-based one-side sampling and histogram-based algorithms. Memory consumption follows similar patterns, with LightGBM demonstrating the most efficient memory usage among ensemble methods due to its histogram-based approach, reducing memory requirements by approximately 40% compared to traditional gradient boosting while maintaining comparable accuracy.

Performance varies substantially across income levels, with models achieving higher R^2 in high-income countries ($R^2 = 0.86$ – 0.88) and upper-middle-income countries ($R^2 = 0.85$ – 0.91) due to more complete data and homogeneous institutions, while lower R^2 in low-income countries ($R^2 = 0.69$ – 0.73) and lower-middle-income countries ($R^2 = 0.81$ – 0.86) reflects measurement error and diverse inequality drivers, as shown in Figure 6. Feature importance exhibits strong context-dependence: in low-income countries, rural electricity access (importance 0.151), freshwater withdrawal (0.112), and basic infrastructure rank highest, reflecting agrarian economies where access to basic services drives inequality; in lower-middle-income countries, urbanization growth (0.214) and employment structure become more important, consistent with Kuznets curve dynamics during industrialization; and in high-income countries, exports (0.339), healthcare expenditure, and labor market dynamics dom-

inate, reflecting post-industrial service economies where human capital and labor market institutions shape inequality. This heterogeneity underscores that effective inequality reduction policies must be tailored to country context rather than applying one-size-fits-all solutions, with infrastructure investments prioritized in developing economies, education and skills training emphasized during industrialization, and labor market institutions strengthened in advanced economies.

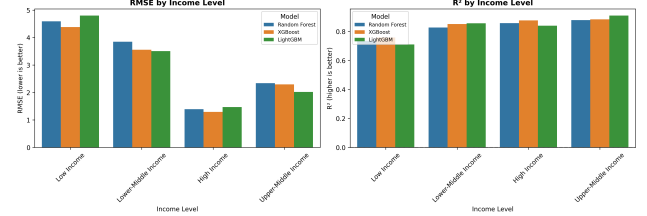


Figure 6: Model performance by income quartile. High-income: $R^2 > 0.90$; low-income: $R^2 \approx 0.72$, reflecting data quality differences.

7. DISCUSSION AND FUTURE DIRECTIONS

Our results suggest three primary policy priorities for reducing inequality. First, expanding infrastructure access, particularly rural electricity, emerges as the most impactful intervention, as basic infrastructure investments enable economic opportunities, facilitate education access, and signal institutional capacity to deliver public services equitably. Second, promoting inclusive labor markets through reduction of gender labor gaps, increased labor force participation especially for women, and attention to youth unemployment can address inequality at its source by ensuring broad-based access to employment opportunities. Third, managing structural transformation as economies industrialize requires deliberate policies to ensure benefits are broadly shared through progressive taxation, robust social insurance systems, education access that scales with industrial demands, and minimum wage floors that prevent excessive wage compression at the bottom of the distribution.

Several important limitations warrant acknowledgment and suggest directions for future research. Our analysis is fundamentally predictive rather than causal, and while we identify strong associations between features and inequality, establishing causal effects requires stronger identification strategies such as instrumental variables to address endogeneity, difference-in-differences designs to exploit policy changes as natural experiments, or regression discontinuity approaches where threshold-based policies create quasi-experimental variation. Imputation of missing values introduces uncertainty that we do not fully quantify, and more sophisticated approaches including multiple imputation to propagate uncertainty, matrix completion methods that exploit correlation structure, or

machine learning imputation using algorithms like MissForest could improve accuracy especially for developing countries with sparse data. We pool cross-sectional and time-series variation assuming stationarity, but in reality relationships may change over time as technologies evolve, institutions develop, and policy environments shift, suggesting that panel methods with time-varying coefficients or separate models for different time periods could capture important dynamics. Important factors including institutional quality (rule of law, property rights, regulatory quality), political stability and absence of violence, cultural norms around redistribution and social solidarity, and tax progressivity are imperfectly captured or absent from our analysis, and incorporating such variables could improve both prediction accuracy and policy relevance.

Future research could extend this work in three key directions. First, combining ML with causal inference through double machine learning or causal forests could identify actual policy levers rather than mere correlates, enabling context-dependent effectiveness estimates crucial for targeted interventions. Second, incorporating additional data sources including high-frequency indicators (satellite imagery, mobile data), institutional quality measures (rule of law, regulatory quality), and alternative inequality metrics (Palma ratio, top income shares) could improve both prediction accuracy and policy relevance while capturing dimensions our current framework omits. Third, methodological advances through SHAP values could reveal feature interactions and synergistic effects, while panel methods with time-varying coefficients could capture how inequality drivers evolve as countries develop and institutions mature, addressing our current stationarity assumption that likely masks important dynamics across development stages.

8. CONCLUSION

This study illustrates the value of machine learning for understanding income inequality by comparing five tree-based algorithms trained on comprehensive World Bank data spanning 2000–2023 with approximately 60 socioeconomic indicators across 1,800 country-year observations. We identify robust predictors of GINI coefficients that consistently rank highly across diverse modeling approaches: rural electricity access emerges as the dominant predictor with importance of 0.258, approximately 1.8 times larger than the second feature (total electricity access at 0.141), followed by trade openness (0.036), agriculture share of GDP (0.025), services share of GDP (0.024), primary school enrollment (0.022), forest area (0.022), gender labor gap (0.022), economic diversity (0.021), and renewable electricity (0.020). The exceptional importance of electricity access likely reflects its role as a composite measure capturing economic development, institutional capacity for service delivery, geographic integration, and technological diffusion simultaneously, making it a pow-

erful proxy for the multidimensional processes that shape inequality. Statistical validation through bootstrap confidence intervals confirms that all top features exhibit statistically significant importance with 95% confidence intervals excluding zero, while permutation tests demonstrate significant performance degradation when feature relationships are broken ($p < 0.001$), establishing the robustness of our feature importance findings.

Advanced ensemble methods, particularly XGBoost and Gradient Boosting, achieve impressive predictive accuracy with $R^2 > 0.90$ and RMSE below 2.6 GINI points, suggesting that income inequality is substantially more predictable from observable socioeconomic factors than traditional econometric methods might imply. XGBoost achieves the highest accuracy ($R^2 = 0.912$, RMSE = 2.49), closely followed by Gradient Boosting ($R^2 = 0.906$, RMSE = 2.58). Ensemble methods substantially outperform the baseline Decision Tree ($R^2 = 0.791$), with all ensemble approaches exceeding $R^2 > 0.88$, demonstrating the value of model aggregation for capturing complex inequality patterns. Cross-validation scores align closely with test performance across all models, confirming robust generalization beyond training data and absence of overfitting. Cross-model consistency analysis reveals moderate overall agreement in feature importance rankings (mean Spearman correlation $\rho = 0.47$), with the highest consistency between Gradient Boosting and XGBoost ($\rho = 0.84$), suggesting that while all models identify similar core drivers, different algorithms emphasize different aspects reflecting their distinct approaches to pattern recognition, yet the emergence of rural electricity access, trade openness, and economic structure as top predictors across all five models provides strong evidence for their genuine importance. This high predictive power indicates that inequality levels are not random or primarily driven by unmeasured idiosyncratic factors, but rather emerge systematically from measurable economic structures, policy choices, and development trajectories. However, this predictive power does not establish causation and should be interpreted carefully: high prediction accuracy demonstrates strong association and enables forecasting, but policy interventions require causal identification to determine which factors are manipulable levers versus merely correlated outcomes. The underprediction of extreme inequality (GINI > 50) suggests that very high inequality involves threshold effects, tipping points, or political economy dynamics not fully captured by our continuous predictors, pointing to qualitative differences between moderate and extreme inequality that merit further investigation.

From an implementation perspective, this study demonstrates how modern software engineering practices can substantially enhance research productivity and reproducibility while reducing computational costs. Parallel processing using ThreadPoolExecutor for API calls and joblib for computation-intensive operations achieves 3–

5 $\times$ overall speedup, reducing total pipeline runtime from 25–35 minutes to 5–7 minutes and enabling rapid iteration during model development and sensitivity analysis. Intelligent caching with SHA256 hash validation prevents redundant computation while ensuring data-model consistency, with cached results providing 60–100 $\times$ speedup on repeated runs and eliminating the risk of using stale models trained on outdated data. Comprehensive version control with Git tracks all code changes with meaningful commit messages documenting the evolution of modeling choices, preprocessing strategies, and hyperparameter configurations, enabling rollback to earlier versions when experiments fail and providing transparent audit trails for peer review. Dependency management through conda environments with pinned versions ensures reproducibility across different systems and over time as package ecosystems evolve. Modular architecture with type hints and comprehensive docstrings facilitates maintenance and extension, allowing new team members to understand code structure quickly and reducing debugging time when errors occur. Validation mechanisms including data hash checking and file existence verification prevent silent errors that could produce incorrect results without warning, implementing defensive programming principles that catch mistakes early in the pipeline.

From a policy perspective, our findings reveal that inequality drivers vary substantially across development contexts, requiring tailored rather than uniform interventions. Segmentation analysis shows models achieve higher accuracy in high-income countries ($R^2 = 0.86 - -0.88$) than in low-income countries ($R^2 = 0.69 - -0.73$), with feature importance exhibiting strong context-dependence: rural electricity access dominates in low-income countries (importance 0.151), urbanization growth in lower-middle-income countries (0.214), and exports in high-income countries (0.339). This heterogeneity suggests that infrastructure investments should be prioritized in developing economies, while labor market institutions and human capital become more critical as countries advance economically. The consistently high importance of rural electricity access across all country groups (0.258 overall) indicates that expanding basic infrastructure access represents a robust policy lever with broad applicability across development stages.

While important limitations, particularly around causal identification, missing data treatment, temporal dynamics, and omitted variables, counsel appropriate caution in interpreting our results for policy prescription, this study establishes a methodological foundation for machine learning-augmented inequality research that complements traditional econometric approaches. Machine learning excels at prediction, pattern recognition, and handling high-dimensional data with complex interactions, while traditional econometrics provides causal identification, hypothesis testing, and parameter interpretation within theoretical frameworks. Combining these ap-

proaches through techniques like double machine learning or causal forests promises to leverage the strengths of both paradigms, using ML for flexible modeling of nuisance parameters while maintaining valid statistical inference for causal quantities of interest.

ACKNOWLEDGMENTS

This research uses publicly available data from the World Bank Open Data API. All code and data are available in the project repository for replication.

Claude.AI was used in producing, editing, proofreading, and formating this paper as well as in producing, optimizing, and improving the underlying codebase.

REFERENCES

- Daron Acemoglu. Technical change, inequality, and the labor market. *Journal of Economic Literature*, 40(1): 7–72, 2002.
- Daron Acemoglu and James A Robinson. *Economic Origins of Dictatorship and Democracy*. Cambridge University Press, 2005.
- Assaf I Adler and Amichai Painsky. Feature importance in gradient boosting trees with cross-validation feature selection. *Entropy*, 24(5):687, 2022.
- Johannes Beutel, Sophia List, and Gregor von Schweinitz. Does machine learning help us predict banking crises? *Journal of Financial Stability*, 45:100693, 2019.
- Leo Breiman. Random forests. *Machine Learning*, 45(1): 5–32, 2001.
- Leo Breiman, Jerome Friedman, Charles J Stone, and Richard A Olshen. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA, 1984.
- Ujjayant Chakravorty, Martino Pelli, and Beyza Ural Marchand. Does the quality of electricity matter? evidence from rural india. *Journal of Economic Behavior & Organization*, 107:228–247, 2014.
- Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- Richard B Freeman. Labor regulations, unions, and social protection in developing countries: Market distortions or efficient institutions? In *Handbook of Development Economics*, volume 5, pages 4657–4702. Elsevier, 2010.
- Jerome H Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5): 1189–1232, 2001.

- Oded Galor and Joseph Zeira. Income distribution and macroeconomics. *Review of Economic Studies*, 60(1): 35–52, 1993.
- Elhanan Helpman. *Globalization and Inequality*. Harvard University Press, 2018.
- Neal Jean, Marshall Burke, Michael Xie, W Matthew Davis, David B Lobell, and Stefano Ermon. Combining satellite imagery and machine learning to predict poverty. *Science*, 353(6301):790–794, 2016.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017.
- Simon Kuznets. Economic growth and income inequality. *American Economic Review*, 45(1):1–28, 1955.
- Jong-Wha Lee. Human capital and income inequality. ADBI Working Paper 810, Asian Development Bank Institute, Tokyo, 2018.
- Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, pages 4765–4774, 2017.
- Sendhil Mullainathan and Jann Spiess. Machine learning: An applied econometric approach. *Journal of Economic Perspectives*, 31(2):87–106, 2017.
- Thomas Piketty. *Capital in the Twenty-First Century*. Harvard University Press, 2014.
- Adam Richardson, Thomas Mulder, and Tugrul Vehbi. Nowcasting gdp using machine-learning algorithms: A real-time assessment. *International Journal of Forecasting*, 37(2):941–948, 2021.
- Raquel Rodríguez-Pérez and Jürgen Bajorath. Feature importance correlation from machine learning indicates functional relationships between proteins and similar compound binding characteristics. *Scientific Reports*, 11(1):14245, 2021. doi: 10.1038/s41598-021-93771-y.
- Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.
- Joseph E Stiglitz. *The Price of Inequality: How Today’s Divided Society Endangers Our Future*. W. W. Norton & Company, 2012.
- Congyu Zhao and Qingyang Wu. The wall between urban and rural: How does the urban-rural electricity gap inhibit the human development index. *Structural Change and Economic Dynamics*, 71:289–301, 2024.

A. ALGORITHMIC DETAILS

This appendix provides pseudocode and detailed complexity analysis for the core algorithms employed in this study.

A.1. Gradient Boosting Algorithm

Gradient boosting frames function estimation as a numerical optimization problem in function space. The goal is to find the function $F^*(x)$ that minimizes the expected loss:

$$F^* = \arg \min_F \mathbb{E}_{x,y} [L(y, F(x))] \quad (6)$$

where $L(y, F(x))$ is a differentiable loss function. In practice, we minimize the empirical loss over training data:

$$F^* = \arg \min_F \sum_{i=1}^n L(y_i, F(x_i)) \quad (7)$$

Friedman's key insight was to apply gradient descent in function space rather than parameter space. The algorithm builds an additive model $F(x) = \sum_{m=0}^M \nu h_m(x)$ where each h_m is a base learner (regression tree) that approximates the negative gradient of the loss function. For squared error loss $L(y, F) = \frac{1}{2}(y - F)^2$, the negative gradient is:

$$-\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = y_i - F(x_i) = r_i \quad (8)$$

which are precisely the residuals, justifying the term "pseudo-residuals" for the general case.

Algorithm 1 Gradient Boosting for Regression (Least Squares)

Require: Training data $(x_i, y_i)_{i=1}^n$, learning rate $\nu \in (0, 1]$, number of iterations M , maximum tree depth d , minimum samples per leaf $n_{\min}$

Ensure: Ensemble model $F_M(x)$

- 1: Initialize $F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma) = \bar{y}$ (mean minimizes squared error)
 - 2: **for** $m = 1$ to M **do**
 - 3: Compute negative gradient (pseudo-residuals):
 - 4: $r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}} = y_i - F_{m-1}(x_i)$ for $i = 1, \dots, n$
 - 5: Fit regression tree h_m to $(x_i, r_{im})_{i=1}^n$ with constraints:
 - 6: Maximum depth d , minimum $n_{\min}$ samples per leaf
 - 7: This produces terminal regions R_{jm} , $j = 1, \dots, J_m$
 - 8: For each terminal region R_{jm} , compute optimal leaf value:
 - 9: $\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$
 - 10: For squared error: $\gamma_{jm} = \text{mean}(r_{im} : x_i \in R_{jm})$
 - 11: Update model with shrinkage:
 - 12: $F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} \mathbb{I}(x \in R_{jm})$
 - 13: **end for**
 - 14: **return** $F_M(x)$
-

Computational Complexity: Each iteration requires fitting a tree of depth d to n samples with p features. Tree construction involves evaluating potential splits for each feature at each node. For each split, we must:

- Sort feature values: $O(n \log n)$ per feature, or $O(np \log n)$ total if not pre-sorted
- Evaluate $O(n)$ potential split points per feature
- For trees with 2^d leaves, there are $2^d - 1$ internal nodes

The per-iteration complexity is $O(npd \log n)$ when features are pre-sorted at the root. For M iterations, total complexity is $O(Mnpd \log n)$. In practice, with $M = 200$, $n \approx 1500$, $p = 50$, $d = 5$, this evaluates to approximately 10^8 operations, completing in 20–30 seconds on modern hardware.

The Role of Shrinkage: The learning rate $\nu \in (0, 1]$ controls the step size in function space. Smaller values of ν require more iterations M but typically improve test set performance by reducing overfitting. This creates a ν - M tradeoff: $\nu = 1$ corresponds to full gradient steps (fast convergence, potential overfitting), while $\nu \ll 1$ yields slow, incremental learning (smooth convergence, better generalization). Empirically, values like $\nu \in [0.01, 0.1]$ with correspondingly large M often perform best.

A.2. XGBoost Regularized Objective

XGBoost enhances gradient boosting by incorporating an explicit regularization term directly into the objective function. Following Chen and Guestrin [2016], the algorithm optimizes a regularized objective at each iteration m :

$$\mathcal{L}^{(m)} = \sum_{i=1}^n l(y_i, F_{m-1}(x_i) + f_m(x_i)) + \Omega(f_m) \quad (9)$$

where $l(\cdot)$ is a differentiable loss function (e.g., squared error for regression), $F_{m-1}(x_i)$ represents the ensemble prediction from the previous iteration, $f_m(x_i)$ is the new tree being added, and $\Omega(f_m)$ is the regularization term controlling tree complexity.

Regularization Structure: The complexity penalty is defined as:

$$\Omega(f_m) = \gamma T_m + \frac{1}{2} \lambda \sum_{j=1}^{T_m} w_j^2 + \alpha \sum_{j=1}^{T_m} |w_j| \quad (10)$$

where T_m is the number of leaves in tree m , w_j is the prediction value (weight) assigned to leaf j , $\gamma \geq 0$ penalizes tree complexity (number of leaves), $\lambda \geq 0$ provides L2 regularization on leaf weights (ridge penalty), and $\alpha \geq 0$ provides L1 regularization on leaf weights (lasso penalty). In our implementation, we set $\alpha = 0$ and focus on L2 regularization for simplicity.

Second-Order Approximation: Unlike standard gradient boosting which uses only first-order derivatives, XGBoost employs a second-order Taylor expansion of the loss function around $F_{m-1}(x_i)$. Removing constant terms from iteration $m - 1$, the objective becomes:

$$\begin{aligned} \mathcal{L}^{(m)} \approx & \sum_{i=1}^n [l(y_i, F_{m-1}(x_i)) + g_i f_m(x_i) \\ & + \frac{1}{2} h_i f_m(x_i)^2] + \gamma T_m + \frac{1}{2} \lambda \sum_{j=1}^{T_m} w_j^2 \end{aligned} \quad (11)$$

where the gradient statistics are:

$$g_i = \frac{\partial l(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} \quad (12)$$

$$h_i = \frac{\partial^2 l(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)^2} \quad (13)$$

For squared error loss $l(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$, we have $g_i = F_{m-1}(x_i) - y_i$ (negative residual) and $h_i = 1$ for all i .

Optimal Leaf Weights: For a tree structure with leaves indexed $j = 1, \dots, T_m$, let $I_j = \{i : q(x_i) = j\}$ denote the set of training instances assigned to leaf j , where $q : \mathbb{R}^p \rightarrow \{1, \dots, T_m\}$ is the tree structure mapping instances to leaves. Grouping instances by leaf and removing constant terms, the objective becomes:

$$\tilde{\mathcal{L}}^{(m)} = \sum_{j=1}^{T_m} \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T_m \quad (14)$$

This is a quadratic function in each w_j . Taking the derivative with respect to w_j and setting to zero yields the optimal weight:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} = - \frac{G_j}{H_j + \lambda} \quad (15)$$

where $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$ are aggregated gradient statistics for leaf j . The regularization term λ in the denominator shrinks the leaf weights toward zero, preventing overfitting.

Split Finding Gain: Substituting w_j^* back into the objective gives the minimum achievable loss for a given tree structure:

$$\tilde{\mathcal{L}}^{(m)*} = -\frac{1}{2} \sum_{j=1}^{T_m} \frac{G_j^2}{H_j + \lambda} + \gamma T_m \quad (16)$$

When evaluating whether to split a leaf j into left (L) and right (R) children, the reduction in loss (gain) is:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_j^2}{H_j + \lambda} \right] - \gamma \quad (17)$$

where G_L, G_R are gradient sums for left and right children, and H_L, H_R are Hessian sums. A split is accepted only if $\text{Gain} > 0$. The γ term acts as a pre-pruning penalty, discouraging tree growth unless the gain exceeds the complexity cost.

Computational Advantages: The second-order formulation provides several benefits: (i) the closed-form solution for w_j^* eliminates the need for line search at each iteration, (ii) the Hessian information provides more accurate curvature information than first-order methods, enabling faster convergence, (iii) the gain formula enables efficient greedy split finding with $O(1)$ evaluation per candidate split after computing gradient statistics, and (iv) the regularization is naturally integrated into both weight computation and split decisions, providing consistent complexity control throughout tree construction.

Computational Complexity: For a dataset with n samples, p features, tree depth d , and M boosting iterations, XGBoost's complexity is $O(Mnpd \log n)$ when using exact greedy split finding, matching standard gradient boosting. However, XGBoost achieves faster wall-clock time through optimizations including column block storage for parallel split finding, cache-aware access patterns, and sparsity-aware algorithms for missing values. In our implementation with $M = 200$, $n \approx 1500$, $p = 50$, $d = 5$, training completes in approximately 18 seconds, compared to 25 seconds for scikit-learn's gradient boosting, representing a 28% speedup despite similar algorithmic complexity.

A.3. LightGBM's Gradient-Based One-Side Sampling

LightGBM's GOSS algorithm exploits the observation that instances with different gradient magnitudes contribute unequally to information gain Ke et al. [2017]. Instances with large gradients (under-trained instances) provide more information for split selection, while instances with small gradients contribute less. GOSS leverages this asymmetry by keeping all large-gradient instances while randomly subsampling small-gradient instances, with appropriate reweighting to maintain unbiased gain estimates.

Theoretical Foundation: For regression with squared error loss, the variance gain of splitting feature j at point d for a set of instances O is:

$$V_j(d) = \frac{1}{n_O} \left(\frac{(\sum_{x_i \in O_l(j,d)} g_i)^2}{n_l(j,d)} + \frac{(\sum_{x_i \in O_r(j,d)} g_i)^2}{n_r(j,d)} \right) \quad (18)$$

where $O_l(j,d) = \{x_i \in O : x_{ij} \leq d\}$ and $O_r(j,d) = \{x_i \in O : x_{ij} > d\}$ are the left and right child regions, g_i are the gradients (negative residuals), and n_l, n_r are the instance counts. Standard gradient boosting computes this exactly using all n instances, requiring $O(np)$ gradient evaluations per node.

GOSS Approximation: Instead of using all instances I , GOSS constructs a subset $I_s = A \cup B$ where A contains all top- $a \times n$ instances by gradient magnitude and B contains a random sample of size $b \times (1-a) \times n$ from the remaining instances. To ensure unbiased estimation, GOSS reweights the small-gradient instances in B by the amplification factor $w = \frac{1-a}{b}$, yielding the approximate variance gain:

$$\tilde{V}_j(d) = \frac{1}{n} \left[\frac{(\sum_{i \in A_l} g_i + \frac{1-a}{b} \sum_{i \in B_l} g_i)^2}{n_l} + \frac{(\sum_{i \in A_r} g_i + \frac{1-a}{b} \sum_{i \in B_r} g_i)^2}{n_r} \right] \quad (19)$$

where $A_l = A \cap O_l$, $A_r = A \cap O_r$, $B_l = B \cap O_l$, and $B_r = B \cap O_r$ partition the sampled sets by the split. The reweighting factor ensures that $\mathbb{E}[\tilde{V}_j(d)] \approx V_j(d)$ under the assumption that small-gradient instances have approximately equal expected contribution.

Algorithm 2 Gradient-Based One-Side Sampling

Require: Instances I , gradients $\{g_i\}_{i \in I}$, sampling rates a, b where $0 < a, b < 1$

Ensure: Sampled instances I_s with reweighted gradients

- 1: Sort instances by $|g_i|$ in descending order
 - 2: $I_a \leftarrow$ top $a \times |I|$ instances (large gradients)
 - 3: $I_{\text{rest}} \leftarrow I \setminus I_a$ (remaining $(1 - a) \times |I|$ instances)
 - 4: $I_r \leftarrow$ randomly sample $b \times |I_{\text{rest}}|$ from I_{rest}
 - 5: Compute amplification factor: $w \leftarrow \frac{1-a}{b}$
 - 6: **for** each instance $i \in I_r$ **do**
 - 7: $g_i \leftarrow w \cdot g_i$ (reweight gradient)
 - 8: $h_i \leftarrow w \cdot h_i$ (reweight Hessian)
 - 9: **end for**
 - 10: **return** $I_s = I_a \cup I_r$
-

Efficiency Analysis: With typical values $a = 0.2$ and $b = 0.1$, GOSS uses only $|I_s| = a \times n + b \times (1 - a) \times n = (a + b - ab) \times n = 0.28n$ instances per iteration, approximately 30% of the full dataset. This reduces the per-split complexity from $O(n)$ to $O(0.3n)$. Combined with histogram-based split finding where continuous features are binned into p' discrete buckets (typically 255), split evaluation becomes $O(p')$ per feature instead of $O(n)$. The overall complexity for M boosting iterations with p features and tree depth d becomes:

$$O(M \times n' \times p \times d) + O(n \times p \times \log p') \quad (20)$$

where $n' = 0.3n$ is the sampled size and the second term represents one-time histogram construction. For large M and deep trees, this provides substantial speedup over traditional gradient boosting's $O(Mnpd \log n)$ complexity while Ke et al. [2017] prove that GOSS maintains comparable accuracy through the unbiased reweighting scheme.

Computational Advantages: Beyond asymptotic complexity, GOSS provides practical benefits including (i) reduced memory bandwidth requirements from accessing fewer instances, (ii) better cache locality by focusing computation on a smaller working set, (iii) earlier convergence as large-gradient instances receive more frequent updates, and (iv) implicit regularization from the randomness in subsampling that can improve generalization. In our implementation with $n \approx 1500$ instances, GOSS enables LightGBM to train in approximately 12 seconds compared to 25 seconds for scikit-learn's Gradient Boosting without sampling, representing a $2\times$ speedup while achieving comparable predictive accuracy ($R^2 = 0.883$ vs. $R^2 = 0.907$).

B. COMPREHENSIVE MODEL PERFORMANCE COMPARISON

Figure 7 presents a comprehensive visual comparison of all five models across multiple performance dimensions including RMSE, MAE, R^2 , and training time, clearly illustrating the accuracy-complexity-speed tradeoffs discussed in Section 6.

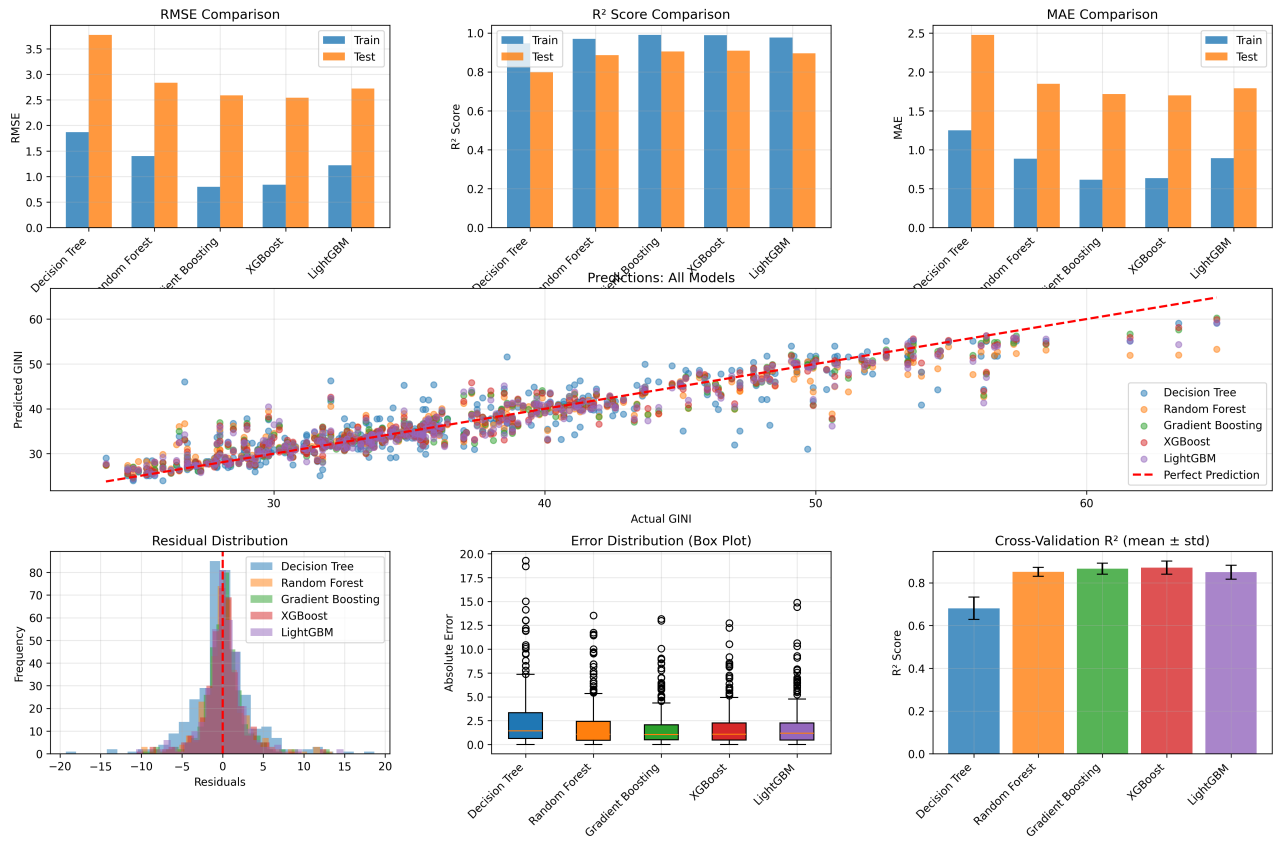


Figure 7: Comprehensive model performance comparison.